

rollout_engine

Helix OTA — 1.0.1 Staged-Rollout Engine Specification

Field	Value
Revision	1
Created	2026-06-08
Last modified	2026-06-08
Status	active (closes gap G7 in research/additions_synthesis.md §8)
Status summary	Full specification of the Helix OTA staged-rollout engine for phase 1.0.1: percentage phases (5/10/30/50/100) gated by success/error thresholds with pause/halt-on-breach; the lifecycle operations (start/pause/resume/abort/rollback); deterministic cohort assignment; and the control-plane REST surface (Gin) that drives the reusable ota-rollout-engine brick. The engine itself is reused, not reinvented — this spec wires the existing brick (submodules/ota-rollout-engine) and adds the control-plane layer the brick deliberately does not provide (HTTP, persistence, pause/resume/abort/rollback). Companion DDL: migration_002_design.md. The brick exposes Create/Start/Evaluate and a terminal-state model (pending/active/halted/held/completed) but has no native pause/resume/abort/rollback verbs — those are control-plane responsibilities layered over the brick's storage port (§5). Health-verdict aggregation (telemetry → HealthVerdict) is owned by telemetry_processing, referenced here, not specified here.
Issues	

Field	Value
	TUF metadata storage is deferred (note in §11).
Issues summary	Engine reused as-is; lifecycle verbs the brick lacks are specified as a thin control-plane state machine over the brick, not as brick changes.
Fixed	Promotes the 1.0.1-staged-rollout/README.md §2 seed into a full, code-actionable engine spec reflecting the brick's real Go API as built in submodules/ota-rollout-engine (types.go, engine.go, verdict.go, decide.go, cohort.go, ports.go).
Fixed summary	Spec body now matches the shipped brick API exactly; no invented engine functions.
Continuation	Implement the control-plane rollout package wiring the brick to a database-backed StoragePort (§6), the scheduler that drives Evaluate (§7), and the REST routes (§8); land migration 002_* (migration_002_design.md); four-layer + mutation tests per §10.
Owner	Helix OTA control-plane team
Related	migration_002_design.md ; README.md ; ../1.0.0-mvp/api/endpoints.md ; ../1.0.0-mvp/database/migrations/001_initial_schema.up.sql ; ../research/additions_synthesis.md (§8 G7, §10 K6/K8); submodules/ota-rollout-engine

table of contents

- [§1. purpose and scope](#)
- [§2. reused brick — real api surface](#)
- [§3. percentage phases and thresholds](#)
- [§4. deterministic cohort assignment](#)
- [§5. lifecycle operations](#)
- [§6. persistence — storage port over postgres](#)
- [§7. evaluation loop and health verdict](#)
- [§8. rollout rest api \(gin\)](#)
- [§9. decoupling and catalogue reuse](#)
- [§10. testing \(four-layer\)](#)
- [§11. tuf metadata note \(deferred\)](#)
- [§12. anti-bluff / unverified register](#)

§1. purpose and scope

Phase 1.0.1 turns the MVP's all-targets, all-at-once delivery into a **safe, staged rollout**: a release reaches the fleet in ordered percentage phases, each gated on telemetry-derived success/error thresholds, with **automatic halt-on-breach** and operator lifecycle controls (start/pause/resume/abort/rollback). This is the phase where the ota-rollout-engine submodule graduates from scaffold to wired-in (README.md §1).

Per additions_synthesis.md §10 K6/K8, **staged rollout owns 1.0.1** (rollback → 1.0.2, delta → 1.0.3) and the MVP is all-targets-only; nothing in this spec changes the MVP contract. The MVP already reserves the seam: deployments.rollout_strategy is JSONB defaulting to {"mode":"all_at_once"} (migration 001), and the deployment API reserves a strategy field (endpoints.md §11). A staged deployment writes a staged strategy and populates the new tables defined in migration_002_design.md.

Reuse mandate (Constitution §11.4.74 catalogue-first, UNVERIFIED clause): the decision core — phase model, threshold gating, halt-wins safety invariant, deterministic cohorts, idempotent transitions — is the existing brick. This spec **does not re-specify those**; it specifies only the control-plane layer the brick intentionally omits (it is HTTP-free and storage-agnostic by design, README.md “Boundary”).

In scope: phase/threshold semantics as the brick implements them; deterministic cohort assignment as the brick implements it; the lifecycle verbs the control plane adds over the brick; the Postgres-backed StoragePort; the evaluation scheduler; the REST API. Out of scope: telemetry aggregation into a HealthVerdict (owned by telemetry_processing); device-side TUF (README.md §4); end-user rollback UX (1.0.2, README.md §5 — this spec's “rollback” is the server-driven recall-to-previous-release operation, §5.5).

§2. reused brick — real api surface

The brick lives at submodules/ota-rollout-engine (Go module github.com/HelixDevelopment/ota-rollout-engine, Apache-2.0). The control plane imports it; it **MUST NOT** fork or reimplement it. The surface below is transcribed from the **shipped source** (not invented).

§2.1 types (types.go)

```
type Phase struct {
    Percentage      int           // cumulative cohort %,
                  // strictly increasing, in (0,100]; last must be 100
    SuccessThreshold float64        // success_rate (fraction
                  // [0,1]) required to advance
    ErrorThreshold  float64        // error_rate (fraction [0,1])
                  // that triggers HALT
    Duration        time.Duration // evaluation window; 0 = "no
                  // time bound", judged purely on thresholds
```

```

    AutoProgress    bool           // true = auto-advance on
                                success; false = HOLD for operator
}

type Status string // "pending" | "active" | "halted" | "held" |
                    "completed"

type State struct {
    DeploymentID string // rollout key (== StoragePort key)
    Phases        []Phase // immutable validated plan
    CurrentPhase  int      // cursor into Phases
    Status        Status
    PhaseStartedAt time.Time // set by engine via Clock
    UpdatedAt     time.Time
}

```

Status values (from types.go): StatusPending, StatusActive, StatusHalted, StatusHeld, StatusCompleted. StatusHalted is documented in-brick as “safety-critical; never auto-resumes”.

Plan validation (validatePhases) rejects: empty phase list (ErrNoPhases); a percentage outside (0,100] (ErrPercentageRange); non-strictly-increasing percentages (ErrPercentageNotMonotonic); a threshold outside [0,1] (ErrThresholdRange); a negative duration (ErrDurationNegative); and a final phase that is not 100 (ErrFinalPercentageNot100). The control plane MUST surface these as 400 VALIDATION_FAILED (§8.2), mapping each sentinel to a details entry.

§2.2 engine + ports (engine.go, ports.go)

```

func New(store StoragePort, clock Clock) (*Engine, error)

func (e *Engine) Create(ctx, deploymentID string, phases []Phase)
    (State, error)
func (e *Engine) Start(ctx, deploymentID string) (State, error)
func (e *Engine) Evaluate(ctx, deploymentID string, v
    HealthVerdict) (Decision, error)

type StoragePort interface {
    Load(ctx, deploymentID string) (State, error) // ErrNotFound
    (wrapped) when absent
    Save(ctx, state State) error
}

type Clock interface { Now() time.Time }
func NewSystemClock() Clock

```

The engine is “stateless beyond its ports”: all rollout state lives behind StoragePort; the only clock is the injected Clock. Create is idempotent-overwrite; Start is idempotent on an already-active rollout and errors from a terminal state. **The brick provides no Pause, Resume, Abort, or Rollback method** — see §5.

§2.3 verdict + decision (verdict.go, decide.go)

```
type HealthVerdict struct {
    SuccessRate    float64 // [0,1] = count(success)/
               count(terminal)
    ErrorRate      float64 // [0,1] = count(failure)/
               count(terminal)
    PostBootHealthFailed
    bool          // post-boot health-window breach -> abort
}

type Decision struct {
    Action         Action // "halt" | "advance" | "hold" |
               "complete"
    Reason         Reason
    Status         Status
    DeviceStatus   otaprotocol.DeviceDeploymentStatus // reuses the
               shared per-device enum
}
```

Decision.DeviceStatus reuses ota-protocol's DeviceDeploymentStatus (pending/downloading/installing/verifying/success/failed/rolled_back) so the rollout speaks the same per-device vocabulary as migration 001's device_deployments.status.

The decision precedence (the **safety invariant**, from decide.go, in order) is:

1. PostBootHealthFailed → **HALT** (ReasonPostBootFailed).
2. ErrorRate >= ErrorThreshold → **HALT** (ReasonErrorThreshold) — checked **before** the success path, so a simultaneous error+success breach halts. “Halt wins over advance.”
3. SuccessRate >= SuccessThreshold → final phase **COMPLETE**; else AutoProgress **ADVANCE** (reset phase clock); else **HOLD** (ReasonAutoProgressOff, → StatusHeld).
4. window still open (or Duration == 0) → **HOLD** (ReasonWindowOpen, stays active).
5. window elapsed, bar not met → **HOLD** (ReasonWindowExpired, → StatusHeld).

Threshold comparisons are >= (a rate exactly at the threshold is a breach / a pass). The control plane **MUST NOT** re-derive or override this ordering; it consumes Decision.

§3. percentage phases and thresholds

A staged rollout is an ordered []Phase. The canonical default plan (synthesis §3, README.md §1) is **5** → **10** → **30** → **50** → **100**, each cumulative. Because Percentage is cumulative and strictly increasing and cohort membership is monotonic (§4), phase *N*'s cohort is a superset of phase *N-1*'s — the rollout only ever grows the exposed fleet.

Example plan (the API request shape is §8.2):

Phase	Percentage (cumulative)	success_threshold	error_threshold	duration	auto_progress
1	5	0.95	0.02	6h	true
2	10	0.95	0.02	6h	true
3	30	0.97	0.01	12h	true
4	50	0.98	0.01	12h	false (operator gate)
5	100	0.98	0.01	24h	true

Thresholds are **fractions in [0, 1]** (not percents) — enforced by `validatePhases`. The default numeric values above are **operator-tunable defaults, UNVERIFIED as production constants** (they are not measured; set them per release from the config brick). `duration` is the evaluation window; `0` means “no time bound — hold until the success bar is met”. `auto_progress=false` on phase 4 illustrates an operator checkpoint: the engine reaches held with `ReasonAutoProgressOff` and waits for an explicit resume (§5.4).

The brick **requires the final phase to be 100** (`ErrFinalPercentageNot100`): a staged rollout must be able to converge on the whole fleet. A plan that should stop short of 100 is not expressible as a single rollout; the operator aborts (§5.5) instead.

§4. deterministic cohort assignment

Cohort membership is the brick’s `cohort.go` (`InCohort`), reused verbatim:

```
func InCohort(deviceID, deploymentID string, cumulativePercentage
             int) bool
```

A device is in the cohort iff its **stable bucket** is strictly less than the cumulative percentage:

```
bucket(deviceID, deploymentID) = FNV-1a(deviceID || 0x00 ||
deploymentID) mod 100
member ⇔ bucket < cumulativePercentage
```

Properties the control plane relies on (from the brick docs):

- **Deterministic across processes/architectures** — FNV-1a (stdlib hash/fnv), not Go’s randomized map hash, so every server computes the same bucket for the same device.
- **Stable** — a pure function of `(deviceID, deploymentID)`; never depends on time, phase, or call order. A device stays in its cohort across re-evaluation (no flapping in/out).
- **Monotonic** — member at percentage $p \Rightarrow$ member at every $q \geq p$. This is what makes the $5 \rightarrow 10 \rightarrow 30 \rightarrow 50 \rightarrow 100$ progression only ever add devices.
- The salt is the **deployment id**, so the same device lands in different buckets for different rollouts (no systematic always-canary device across releases).
- `cumulativePercentage <= 0` selects nobody; `>= 100` selects everybody.

Where it is used. On a device update-check (endpoints.md §12.1), for a device whose release is governed by a staged deployment, the control plane:

1. loads the rollout State (§6) and reads CurrentPhase → cumulative Percentage;
2. calls InCohort(deviceID, deploymentID, percentage);
3. if **true** and the rollout Status is active, offers the update (200); if **false**, or the rollout is pending/halted/held, returns 204 (not yet in cohort / not eligible).
The existing anti-downgrade invariant (synthesis §8 G1) still applies on top.

Cohort assignment is read-only and side-effect-free — it never writes state, so the update-check stays cheap and the cohort is recomputed identically on every poll.

§5. lifecycle operations

The operator-facing verbs are **start** / **pause** / **resume** / **abort** / **rollback**. The brick natively provides only Create, Start, and Evaluate with the terminal-state model pending/active/halted/held/completed. The remaining verbs are a **thin control-plane state machine layered over the brick's StoragePort** — the control plane reads/writes State (and the rollouts row, migration_002_design.md) without forking the brick. This keeps the brick's safety invariant authoritative while adding the operational controls 1.0.1 needs.

Design note (anti-bluff): pause/resume/abort/rollback are **NOT brick methods** (verified against engine.go). Implementing them as control-plane transitions over the persisted State.Status is the chosen design; it does not require brick changes. The mapping below is the spec's decision, not an existing brick API.

§5.1 status model (brick ∪ control-plane)

Status	Source	Meaning	Auto-resumes?
pending	brick	created, not started	n/a
active	brick	a phase is running and being evaluated	n/a
held	brick	window expired below bar, or auto_progress=false — awaits operator	no
halted	brick	error-threshold/post-boot breach (safety)	never
completed	brick	final (100%) phase met its bar	terminal
paused	control plane	operator-suspended (no evaluation advances it)	only via resume
aborted	control plane	operator cancelled the whole rollout	terminal
rolled_back	control plane		terminal

Status	Source	Meaning	Auto-resumes?
		operator recalled the fleet to the previous release	

paused/aborted/rolled_back live in the rollouts table's own status column (migration_002_design.md §3.2), layered over — not inside — the brick State.Status. The control plane is the single writer that reconciles the two (§6.3).

§5.2 start

POST .../rollout/start → Engine.Start(ctx, deploymentID):
 pending → active, stamps PhaseStartedAt. Idempotent on an already-active rollout (brick guarantee). Errors from a terminal status. Sets the deployments.status to active and the rollouts status to active.

§5.3 pause

POST .../rollout/pause: control-plane-only. Allowed from active or held. The control plane sets the rollouts status to paused and **stops feeding verdicts to Evaluate** (the scheduler skips paused rollouts, §7). The brick State is left untouched (no phase clock reset), so the evaluation window is effectively frozen at the wall-clock sense only insofar as the scheduler does not call Evaluate; because the brick judges the window off PhaseStartedAt vs Clock.Now(), **pausing does not extend the brick's window** — on resume a Duration-bounded phase may already be expired and resolve to held. This is the chosen, documented behavior (UNVERIFIED whether operators want true window-freezing; if so, that is a brick change tracked in §12, not assumed here).

§5.4 resume

POST .../rollout/resume: allowed from paused or held.

- From paused: clear paused, restore the underlying brick status (active/held), and re-enable scheduler evaluation.
- From held: the operator decision after an auto_progress=false checkpoint or a window-expired hold. Resume calls Engine.Evaluate once more with the latest verdict; if the success bar is met it advances (or completes). For an operator who wants to advance a held rollout that has **not** met the bar, that is an explicit override and is **out of scope for 1.0.1** (the engine will re-hold) — UNVERIFIED requirement (§12).

§5.5 abort

POST .../rollout/abort: allowed from any non-terminal status. Control-plane-only, terminal: sets rollouts status aborted, deployments.status = cancelled, stops scheduler evaluation, and ceases offering the update to not-yet-updated devices (cohort check returns 204 for an aborted rollout, §4). Devices already updated are **not** reverted — abort halts forward progress only. Reverting them is **rollback** (§5.6). Abort records an entry in rollback_history with kind = 'abort' for audit (migration_002_design.md §3.3).

§5.6 rollback (server-driven recall)

POST .../rollout/rollback: allowed from active, held, halted, paused, or aborted. This is the **server-driven recall-to-previous-release** operation (not end-user rollback, which is 1.0.2). It:

1. terminates the current rollout (rollouts status rolled_back);
2. identifies the **previous good release** for the same os+target_model (the release the target devices were on before this deployment);
3. creates a **new deployment** of that previous release to the affected cohort (reusing the normal deploy path), so devices receive a forward update *back to* N-1. On Android A/B this is a normal forward apply to the prior image; it **MUST** respect the anti-downgrade / rollback-index invariant — a recall is an *authorized* downgrade and requires the prior image's rollback-index to still be acceptable (README.md §5; UNVERIFIED interplay with AVB rollback-index, §12);
4. writes a rollback_history row (kind = 'rollback', from_release_id, to_release_id, deployment_id, triggered_by).

Per §10 K8, full end-user/multi-version rollback is 1.0.2; this 1.0.1 operation is the server-initiated recall only.

§5.7 lifecycle transition table

From Op	start	pause	resume	abort	rollback	evaluate→halt
pending	active	—	—	aborted	rolled_back	—
active	(noop)	paused	—	aborted	rolled_back	halted
held	—	paused	active/ held*	aborted	rolled_back	halted
paused	—	(noop)	active/ held	aborted	rolled_back	— (not evaluated)
halted	—	—	—	aborted	rolled_back	(noop)
completed	—	—	—	—	rolled_back	(noop)
aborted/ rolled_back	—	—	—	—	—	—

* resume from held re-evaluates (§5.4). A blank cell is a 409 CONFLICT (illegal transition, §8). halted **never** auto-resumes; the only ways out are abort or rollback.

§6. persistence — storage port over postgres

§6.1 binding

The control plane implements rollout.StoragePort over the database brick (Postgres, schema helix_ota), backed by the rollouts + deployment_phases tables (migration_002_design.md). New(store, NewSystemClock()) constructs the engine once at startup; the store is request-scoped only insofar as it carries the *sql.Tx/pool.

§6.2 load / save mapping

rollout.State field	Postgres source
DeploymentID	rollouts.deployment_id (the key)
Phases	deployment_phases rows for the deployment, ordered by phase_index → []Phase
CurrentPhase	rollouts.current_phase
Status	rollouts.engine_status (brick status; distinct from the control-plane rollouts.status, §6.3)
PhaseStartedAt	rollouts.phase_started_at
UpdatedAt	rollouts.updated_at

Load returns `rollout.ErrNotFound` (wrapped) when no rollouts row exists for the deployment, per the port contract. Save upserts the rollouts cursor columns in one statement; the immutable deployment_phases rows are written once at create and never updated by Save (the brick treats Phases as immutable).

§6.3 two status columns (engine vs control-plane)

rollouts carries **both** the brick engine_status (pending/active/halted/held/completed) and the control-plane status (adds paused/aborted/rolled_back). The control plane is the **single writer** that keeps them consistent: brick-driven transitions (start, evaluate) write engine_status; operator-driven transitions (pause/abort/rollback) write status; resume reconciles status back to mirror engine_status. The cohort/update-check eligibility check (§4) gates on **both**: a device is eligible iff engine_status = active AND status NOT IN (paused, aborted, rolled_back).

§6.4 concurrency

The brick states concurrency control over a given deployment id is “the storage layer’s responsibility”. The StoragePort implementation **MUST** serialize Load→Evaluate→Save for one deployment with `SELECT ... FOR UPDATE` on the rollouts row (or an equivalent advisory lock) so two concurrent evaluations cannot race the cursor. (UNVERIFIED: whether the database brick exposes a row-lock helper — wire if present, else raw `FOR UPDATE`.)

§7. evaluation loop and health verdict

The brick does not ingest telemetry; it consumes a HealthVerdict per evaluation (`verdict.go`). 1.0.1 adds a **scheduler** that, on a fixed tick (configurable via config, default UNVERIFIED — set from load tests), for each active, non-paused rollout:

1. asks telemetry_processing for the current phase cohort’s HealthVerdict (SuccessRate, ErrorRate, PostBootHealthFailed) computed over the

- cohort's terminal devices (telemetry_processing §4 — referenced, not specified here);
- 2. calls `Engine.Evaluate(ctx, deploymentID, verdict)`;
- 3. acts on the returned `Decision`: on halt raise a Herald alert + mark `deployments.status` accordingly and write a `rollback_history` audit row if policy auto-rollbacks on halt (operator-config, default **halt-and-hold**, not auto-rollback — UNVERIFIED policy default, §12); on advance/complete/hold simply persist (the brick already did, via `Save`).

The scheduler is the **only** caller of `Evaluate` (single writer per §6.4). A device telemetry report does **not** directly drive the engine; it updates `device_deployments` / `telemetry_events`, and the next scheduler tick re-derives the verdict. This decouples the high-volume telemetry path from the rollout decision (Constitution §11.4.28 decoupling).

`Evaluate` is idempotent at a terminal status (halted/completed) — the scheduler may safely re-tick a finished rollout; it writes nothing.

§8. rollout rest api (gin)

Extends the MVP REST conventions (endpoints.md §2–§6): base path `/api/v1`, Gin (gin-gonic), `application/json`; `charset=utf-8`, OAuth2/JWT + RBAC, the Error envelope (`code/message/request_id/details`), Brotli/gzip negotiation, X-Request-Id. New routes hang off the existing deployment resource. **RBAC**: all rollout write verbs require operator; reads require viewer (mirrors endpoints.md §4.2).

§8.1 route summary

Method + path	Op	Min role	Maps to
POST <code>/api/v1/deployments/{id}/rollout</code>	create staged plan	operator	<code>Engine.Create</code>
POST <code>/api/v1/deployments/{id}/rollout/start</code>	start	operator	<code>Engine.Start</code>
POST <code>/api/v1/deployments/{id}/rollout/pause</code>	pause	operator	control-plane (§5.3)
POST <code>/api/v1/deployments/{id}/rollout/resume</code>	resume	operator	control-plane + <code>Engine.Evaluate</code> (§5.4)
POST <code>/api/v1/deployments/{id}/rollout/abort</code>	abort	operator	control-plane (§5.5)
		operator	control-plane (§5.6)

Method + path	Op	Min role	Maps to
POST /api/v1/deployments/{id}/rollout/rollback	server-driven recall		
GET /api/v1/deployments/{id}/rollout	read rollout state	viewer	StoragePort.Load projection

A staged deployment is created with the MVP POST /api/v1/deployments (endpoints.md §11) using strategy: "staged" (the new accepted value alongside all-targets); the rollout plan is then attached with POST .../rollout. Creating the plan before start keeps the deployment in draft/active per the MVP status model.

§8.2 create — POST /api/v1/deployments/{id}/rollout

- **Auth:** operator. Accepts optional Idempotency-Key (mirrors endpoints.md §2).
- **Request body** (RolloutCreate):

```
{
  "phases": [
    { "percentage": 5, "success_threshold": 0.95,
      "error_threshold": 0.02, "duration": "6h",
      "auto_progress": true },
    { "percentage": 10, "success_threshold": 0.95,
      "error_threshold": 0.02, "duration": "6h",
      "auto_progress": true },
    { "percentage": 30, "success_threshold": 0.97,
      "error_threshold": 0.01, "duration": "12h",
      "auto_progress": true },
    { "percentage": 50, "success_threshold": 0.98,
      "error_threshold": 0.01, "duration": "12h",
      "auto_progress": false },
    { "percentage": 100, "success_threshold": 0.98,
      "error_threshold": 0.01, "duration": "24h",
      "auto_progress": true }
  ]
}
```

duration is an RFC-3339-duration-style Go string ("6h", "30m"); the server parses it to time.Duration. success_threshold/error_threshold are fractions in [0,1]. The server calls Engine.Create, which runs validatePhases; each brick sentinel maps to a details entry under 400 VALIDATION_FAILED:

Brick error	details.issue
ErrNoPhases	“at least one phase is required”
ErrPercentageRange	“phase percentage must be in (0,100]”
ErrPercentageNotMonotonic	“phase percentages must strictly increase”

Brick error	details.issue
ErrThresholdRange	“thresholds must be fractions in [0,1]”
ErrDurationNegative	“phase duration must not be negative”
ErrFinalPercentageNot100	“final phase percentage must be 100”

- **Response 201** (Rollout): the persisted plan + engine_status: "pending", status: "pending", current_phase: 0.
- **Status codes:** 201; 200 (idempotent replay); 400 VALIDATION_FAILED; 401/403; 404 NOT_FOUND (deployment); 409 CONFLICT (deployment already has a rollout, or is not a staged deployment); 429.

§8.3 lifecycle verbs — start / pause / resume / abort / rollback

Each is a bodyless POST (optionally { "reason": "..."} for audit). Responses return the updated Rollout projection. Common status codes: 200 OK; 401/403; 404 NOT_FOUND; 409 CONFLICT (illegal transition per §5.7). rollback additionally returns the newly-created recall deployment_id in the body.

```
// Rollout projection (GET and lifecycle responses)
{
  "deployment_id": "d12b...uuid",
  "engine_status": "active",
  "status": "active",
  "current_phase": 2,
  "current_percentage": 30,
  "phase_started_at": "2026-06-08T09:00:00Z",
  "phases": [ /* RolloutCreate phases, with per-phase index */ ],
  "updated_at": "2026-06-08T12:00:00Z"
}
```

§8.4 read — GET /api/v1/deployments/{id}/rollout

- **Auth:** viewer. Returns the Rollout projection (above) from StoragePort.Load.
- **Status codes:** 200; 401/403; 404 NOT_FOUND (no rollout for the deployment).

§8.5 device update-check interaction

The MVP GET /api/v1/client/update (endpoints.md §12.1) gains a staged branch: for a device governed by a staged deployment, the handler applies §4 cohort + §6.3 eligibility before offering 200; otherwise 204. The response body shape is unchanged — staged rollout changes *who* gets the 200, not the contract. No new device-facing fields in 1.0.1.

§9. decoupling and catalogue reuse

Concern	Source (reused)
Phase model, threshold gating, halt-wins, deterministic cohort, idempotent transitions	ota-rollout-engine (the brick — NOT reimplemented)
Per-device status vocabulary	ota-protocol DeviceDeploymentStatus (via Decision.DeviceStatus)
Persistence (StoragePort impl)	database brick → rollouts + deployment_phases (migration_002_design.md)
Health verdict (telemetry → HealthVerdict)	telemetry_processing + ota-telemetry-schema (referenced, owns aggregation)
Halt alerting	Herald
HTTP/transport/auth/limits	Gin + http3/middleware/ratelimiter/auth/security (as endpoints.md §15)
Scheduler tick interval, threshold defaults, lock helper	config brick

Decoupling (Constitution §11.4.28, UNVERIFIED clause): the brick has **no HTTP and no OS specifics** (README.md “Boundary”); this spec keeps it that way — transport, persistence, and telemetry aggregation are all outside the brick, wired by the control plane. The brick stays reusable by other fleet projects.

§10. testing (four-layer)

Per endpoints.md §14 / Constitution §1 (four-layer + mutation). The rollout decision path and the cohort path are **safety-critical** → **≥90% coverage** (synthesis §8 C8/K7).

1. **Source-presence gate.** Each rollout route (§8.1) is registered on the Gin router; the RolloutCreate/Rollout schemas exist in the API contract; the control-plane StoragePort impl satisfies rollout.StoragePort; the brick is imported (not vendored-and-edited).
2. **Artifact gate (bytes shipped).** Boot the server; assert the rollout routes appear in the live route table and migration 002 is applied (the rollouts/deployment_phases/rollback_history tables exist).
3. **Runtime / integration.** End-to-end on the dev stack (monolith + Postgres): create staged deployment → attach 5/10/30/50/100 plan → start → feed verdicts via the scheduler → advance through phases → complete; negative: a verdict with `error_rate >= threshold` **halts** and a halted rollout never auto-resumes; `auto_progress=false` phase reaches held; pause→resume; abort stops offering the update (cohort returns 204); rollback creates the recall deployment + rollback_history row. **Brick-level property tests already exist** (engine_test.go, cohort_test.go, decide_test.go) — the control

plane MUST NOT duplicate them; it tests the wiring (persistence round-trip, REST mapping, eligibility).

4. **Mutation meta-test.** Mutate and assert PASS→FAIL: invert the §4 eligibility (InCohort) so an out-of-cohort device is offered (must fail the cohort test); map a brick halt Decision to “continue” in the scheduler (must fail the halt test); drop the status NOT IN (paused, aborted, rolled_back) clause from §6.3 eligibility (must fail the abort test); reverse a §5.7 illegal-transition 409 (must fail the lifecycle test). The brick’s own halt-wins / deterministic-cohort mutation coverage is its responsibility (README.md).

§11. tuf metadata note (deferred)

Device-side TUF/Uptane trust is **deferred** (README.md §4; ADR-0002; synthesis §10 K2/C5). This engine spec does **not** add TUF metadata tables or fields; the artifact trust model in 1.0.1 remains MVP plain-signing (ed25519 + SHA-256 + AVB) layered byte-identically so TUF drops in later without rework (endpoints.md §1). TUF metadata storage (tuf_metadata / Director+Image repository split) is tracked for a **later 1.0.1 sub-spec or 1.0.2**, and is called out as a deferred, explicitly-not-created concern in migration_002_design.md §5 so the DDL is forward-compatible. Adding TUF must not require changing the rollouts/deployment_phases schema (the rollout engine is trust-model-agnostic).

§12. anti-bluff / unverified register

Per Constitution §11.4.6 (no-guessing) / §7.1 (no-bluff); these MUST NOT be propagated as fact.

- **Pause/resume/abort/rollback are NOT brick methods.** Verified against engine.go: the brick exposes only Create/Start/Evaluate. The lifecycle verbs are a control-plane state machine over the persisted State/rollouts row (§5) — that is this spec’s **design decision**, not an existing API.
- **Pause does not freeze the brick’s evaluation window** (§5.3): the brick judges the window off PhaseStartedAt vs Clock.Now(), which the control plane does not rewind. Whether operators require true window-freezing is **UNVERIFIED**; if required it is a brick change, not assumed here.
- **Threshold/percentage/duration defaults** in §3 (5/10/30/50/100; 0.95–0.98 success; 0.01–0.02 error; 6h/12h/24h) are **operator-tunable defaults, never measured** — set per release from config and MVP load data.
- **Scheduler tick interval** and the **auto-rollback-on-halt policy default** (§7) are **UNVERIFIED** — bind from load tests; the spec’s stated default is halt-and-hold (no auto-rollback).
- **Override-advance of a held rollout that has not met the bar** (§5.4) is **out of scope / UNVERIFIED requirement** for 1.0.1; the engine re-holds.
- **AVB rollback-index interplay with server-driven recall** (§5.6) is **UNVERIFIED** — an authorized recall to N-1 must still satisfy the device’s rollback-index; the exact interplay is the research/stacks/android-avb-rollback.md open item (README.md §5).
- **database brick row-lock / idempotency helper surfaces** (§6.4) are **UNVERIFIED** — wire if present, else raw SELECT ... FOR UPDATE.
- **telemetry_processing §4 verdict-aggregation contract** is referenced, not inspected in this pass — **UNVERIFIED** that it emits exactly {SuccessRate,

ErrorRate, PostBootHealthFailed}; the field set is taken from the brick's HealthVerdict (verified) and the consumer side must match it.

- **HelixConstitution clause numbers** (§11.4.74, §11.4.28, §11.4.6, §7.1, §1) are carried from corpus convention and remain **UNVERIFIED** against the authoritative text (the Constitution file is not present in this repo), consistent with endpoints.md §16.